Check for updates

SOFTWARE TOOL ARTICLE

'dstidyverse': An Implementation of TidyverseWithin the DataSHIELD Ecosystem

[version 1; peer review: 2 approved]

Tim Cadman[1,2], Mariska Slofstra[1], Demetris Avraam[3], Eleanor Hyde[1], Niels Kikkert[1], Marije van der Geest[1], Dick Postma[1], Ruben Veenstra[1], Stuart Wheater[4], Erik Zwart[1], Morris Swertz[1]

[1]Department of Genetics, Genomics Coordination Center, University Medical Centre, Groningen, The Netherlands
[2]Barcelona Institute for Global Health (ISGlobal), Barcelona, Spain
[3]Department of Public Health, University of Copenhagen, Øster Farimagsgade, Copenhagen, Denmark
[4]Newcastle Helix, Urban Science Building, Newcastle upon Tyne, Arjuna Technologies, Newcastle, UK

v1 **First published:** 20 Jun 2025, **14**:606
https://doi.org/10.12688/f1000research.164345.1
Latest published: 20 Jun 2025, **14**:606
https://doi.org/10.12688/f1000research.164345.1

Abstract

Background

DataSHIELD is a mature, R-based federated learning platform that enables multi-site analysis without sharing individual participant data. While DataSHIELD includes many packages for data analysis, it lacks user-friendly data manipulation tools.

Methods

To address this gap, we developed **dsTidyverse**, an implementation of selected functions from the popular Tidyverse package within the DataSHIELD client-server architecture. Disclosure checks were implemented to prevent individual-level data leakage.

Results

This package provides functionality for selecting, renaming, and creating columns; conditional recoding; combining data frames by rows or columns; filtering and arranging rows; grouping and ungrouping data; and converting data frames to tibbles. Through examples, we demonstrate how **dsTidyverse** simplifies common data manipulation tasks within DataSHIELD.

Conclusions

Open Peer Review

Approval Status ✓ ✓

	1	2
version 1 20 Jun 2025	✓ view	✓ view

1. **Miroslav Puskaric**, University of Stuttgart, Stuttgart, Germany
2. **Olaitan I Awe**, African Society for Bioinformatics and Computational Biology, Cape Town, South Africa

Any reports and responses or comments on the article can be found at the end of the article.

By providing additional data manipulation functionality, **dsTidyverse** improves the user experience and analytical efficiency within DataSHIELD. The package is open-source and freely available on CRAN and GitHub, and welcomes further development: https://github.com/molgenis/ds-tidyverse.

Keywords
datashield, federated analysis, R, tidyverse, data manipulation

This article is included in the RPackage gateway.

Corresponding author: Tim Cadman (t.j.cadman@umcg.nl)

Author roles: Cadman T: Conceptualization, Methodology, Project Administration, Software, Writing – Original Draft Preparation, Writing – Review & Editing; **Slofstra M**: Software, Writing – Review & Editing; **Avraam D**: Software, Writing – Review & Editing; **Hyde E**: Project Administration, Writing – Review & Editing; **Kikkert N**: Project Administration, Writing – Review & Editing; **van der Geest M**: Project Administration, Writing – Review & Editing; **Postma D**: Software, Writing – Review & Editing; **Veenstra R**: Software, Writing – Review & Editing; **Wheater S**: Software, Writing – Review & Editing; **Zwart E**: Software, Writing – Review & Editing; **Swertz M**: Conceptualization, Funding Acquisition, Project Administration, Software, Supervision, Writing – Review & Editing

Competing interests: No competing interests were disclosed.

Grant information: This project was funded by the European Union's Horizon Europe programme under grant agreement No. 101137317 (IHENproject). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or European Health and Digital Executive Agency (HADEA). Neither the European Union nor the granting authority can be held responsible for them. Funding was also received from the European Union's Horizon 2020 research and innovation programme under grant agreement No 874583 (ATHLETE).
The funders had no role in study design, data collection and analysis, decision to publish, or preparation of the manuscript.

Copyright: © 2025 Cadman T *et al*. This is an open access article distributed under the terms of the Creative Commons Attribution License, which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited.

How to cite this article: Cadman T, Slofstra M, Avraam D *et al.* ***'dstidyverse': An Implementation of TidyverseWithin the DataSHIELD Ecosystem*** **[version 1; peer review: 2 approved]** F1000Research 2025, **14**:606 https://doi.org/10.12688/f1000research.164345.1

First published: 20 Jun 2025, **14**:606 https://doi.org/10.12688/f1000research.164345.1

1. Introduction

While the analysis of single data sources is a core part of epidemiological research, incorporating data from multiple sources has a number of advantages. These include increased statistical power to detect rare disease outcomes and the opportunity to replicate studies in different populations (Pinot de Moira et al. (2021) Cadman et al. (2024)). Historically, the analysis of multiple data sources has been conducted by either (i) data transfer or (ii) each partner conducting analyses separately and sharing summary statistics. Although both approaches are effective in many situations, they have drawbacks. The physical transfer of data can be restricted by data protection legislation and local data management policies, while requiring each partner to conduct parallel analyses can be time inefficient and inflexible (Knoppers et al. (2011)).

A promising alternative is federated (remote) analysis which does not share individual-level data. Federated analysis allows one researcher to conduct all analyses flexibly, while allowing control of the data to remain with the data owner (Doiron et al. (2013)). One mature implementation of federated analysis is the open-source R-based platform DataSHIELD (Gaye et al. (2014)). DataSHIELD is based on a client-server architecture. In a multisite setting, individual study participants' data are stored on the server of each data source, often protected by a firewall. The data from each site are not directly viewable or accessible to the analyst and cannot be copied or transferred. On the client side, the researcher has access to several DataSHIELD-specific R packages. Using the functions from these packages, the researcher issues analysis commands that are then sent to each server. There are two types of DataSHIELD functions: (i) assign-type functions, which create a new object on the server (e.g., recoding a variable), and (ii) aggregate-type functions, which return summary statistics to the researcher (e.g., means, standard deviations and model parameters). These commands are evaluated on each server, and automated checks are performed to ensure that the operations do not disclose individual-level data.

DataSHIELD has been successfully used in many large European research projects including LifeCycle (researching the role of novel integrated markers of early-life stressors on health across the lifecycle; Jaddoe et al. (2020), Pinot de Moira et al. (2021)) and ATHLETE (understanding and preventing health effects of environmental hazards and their mixtures; Vrijheid et al. (2021)). It has an ever-expanding set of packages supporting a wide range of analyses, including omics, exposure, mediation, survival and machine learning (Escriba-Montagut et al. (2024)).

However, a key weakness of DataSHIELD is that it currently lacks effective functionality to perform basic data manipulation, as most developments have focused on extending the analysis capabilities. Many researchers have complained that it is cumbersome to perform basic operations in DataSHIELD, which would normally be straightforward using R. For example, within DataSHIELD, there are currently limited options to (i) recode variables using if-else style operations, (ii) rename variables, (iii) subset columns by column name, (iv) subset rows by multiple conditions, or (v) group data and perform operations by group.

Complicated workarounds are possible, but these greatly increase computational time and lead to verbose analysis scripts. Consider the example of transforming the continuous variable 'mpg' (miles per gallon) within the 'mtcars' dataset into a 4-level categorical variable (0-15, 15-20, 20-25, >=25). Using the core DataSHIELD package (dsBaseClient), the user is required to first create separate vectors indicating whether participants are above each threshold, which are then added together to create the final variable:

```
ds. Boole(V1 = "mtcars$mpg", V2 = 15, Boolean.operator = ">=", newobj
= "mpg_cat_1")
ds. Boole(V1 = "mtcars$mpg", V2 = 20, Boolean.operator = ">=", newobj
= "mpg_cat_2")
ds. Boole(V1 = "mtcars$mpg", V2 = 25, Boolean.operator = ">=", newobj
= "mpg_cat_3")
ds.assign (expr = "mpg_cat_1 + mpg_cat_2 + mpg_cat_3", newobj = "mpg_category")
```

In contrast, within R outside DataSHIELD, there are many options for efficient data manipulation. One widely used set of packages is the "Tidyverse," which comprises a set of packages for data science that share a common design philosophy, grammar and data structures (Wickham et al. (2019)). These include packages for data manipulation (dplyr), advanced data frames (tibble), and packages for functional programming (purrr) and many others.

Whilst the functionality provided by these packages would greatly improve the user-experience with DataSHIELD, they cannot be used 'off-the-shelf.' They first need to be translated into a bespoke DataSHIELD package using the client-server architecture described above, and additional checks need to be written to ensure that they do not inadvertently

facilitate the leakage of individual participant data. Here, we report the development of dsTidyverse, a DataSHIELD implementation of selected Tidyverse functions available as free open-source software (LGPLv3) at GitHub and the R CRAN.

2. Implementation

2.1 Package structure

As described above, each DataSHIELD package contains two components: a client-side and server-side package. The client-side package is installed locally by the researcher and contains functions called in their analysis scripts. The server-side package is installed on the server with the data and contains functions called by the client-side package. For example, to return the mean of a vector, two functions are required: ds.mean() (client-side, included in the dsBaseClient package) and meanDS() (server-side, included in the dsBase package). When an analyst makes a call to ds.mean(), the following steps occur: (i) arguments are checked for validity on the client-side; (ii) an invocation requesting the calling of the function meanDS() is made via the DataSHIELD Interface (DSI) package which handles API calls to the server; (iii) the request, method and arguments are checked for validity on the server-side; (iv) the server-side function meanDS() calculates the mean and performs checks that this value is not disclosive; and (v) the mean of the vector is returned to the client. Following this architecture we implemented two packages: dsTidyverse and dsTidyverseClient. All code was reviewed by co-author SW (an experienced DataSHIELD developer and maintainer of dsBase) to ensure that it met the DataSHIELD disclosure protection standards.

2.2 Functionality

Given that DataSHIELD functions need to be implemented individually, it is not realistic to implement the entire set of Tidyverse functions. Instead, we reviewed the existing functionality in DataSHIELD and chose those Tidyverse functions that we believed would significantly improve data manipulation within DataSHIELD. Currently, these functions are from the packages dplyr and tibble, although we are open to adding further functions on request and welcoming Github pull requests. The functions implemented at the time of writing are listed in Table 1.

dsTidyverse supports non-standard evaluation (Mailund and Mailund (2018)). The name of the server-side data frame is passed in quotes to df.name, whilst the variable names are passed unquoted and are evaluated as columns within the data frame. Various helper functions can also be used within the 'tidy_expr' argument (for example 'all_of' and 'any_of') to specify multiple variables in filter conditions. See examples at the end of this section on the use of dsTidyverse and the package vignette for a more detailed guide.

Table 1. Implemented Tidyverse functions.

Package	Function	Description
dplyr	select	Choose columns from a data frame.
dplyr	rename	Rename columns in a data frame.
dplyr	mutate	Create or modify columns.
dplyr	if_else	A vectorised conditional function.
dplyr	case_when	A general vectorised conditional function.
dplyr	bind_cols	Combine data frames by columns.
dplyr	bind_rows	Combine data frames by rows.
dplyr	filter	Filter rows based on conditions.
dplyr	slice	Select rows by position.
dplyr	arrange	Arrange rows by values of a column or multiple columns.
dplyr	group_by	Group data by one or more columns.
dplyr	ungroup	Remove grouping from data.
dplyr	group_keys	Retrieve the group keys from a grouped data frame.
dplyr	distinct	Return unique rows based on certain columns.
tibble	as_tibble	Convert data to a tibble.

2.3 Disclosure checks

A key feature of DataSHIELD is the various disclosure checks performed by the server-side package to ensure that individual participant data or any other output that can be used to infer any individual participant information is not returned to the analyst. All but one of the dsTidyverse functions currently implemented are assign-type functions, and these carry a lower risk or direct disclosure, as they do not return anything to the client. However, they carry a risk of indirect exposure, especially in the case of subsetting operations. For example, by creating a subset of data with only one row less than the original data, the summary statistics of the two data frames can be compared to reveal the values of the row in difference. To mitigate against these risks, we implemented the following disclosure checks:

1. We specified a list of permitted functions that can be passed within the 'tidy_expr' argument of assign-type functions calls; non-permitted functions will be blocked. The currently permitted functions are:

```
"everything", "last_col", "group_cols", "starts_with", "ends_with", "contains",
"matches", "num_range", "all_of", "any_of", "where", "rename", "mutate",
"if_else", "case_when", "mean", "median", "mode", "desc", "last_col", "nth",
"where", "num_range", "exp", "sqrt", "scale", "round", "floor", "ceiling", "abs",
"sd", "var", "sin", "cos", "tan", "asin", "acos", "atan", "c".
```

2. We check that the variable names passed within the 'tidy_expr' argument are not longer than a specified parameter to reduce the risk of malicious code being passed.

3. To guard against subsetting attacks (malicious attempts to infer individual-level data by taking subsets of data), we check that no subsets are created (e.g. by ds.filter()) with (i) the number of rows lower than a specified parameter or (ii) with the difference between the number of rows of the original dataset and the subset dataset less than a given parameter.

4. We check that the output from 'ds.group_keys' (the groups in a grouped data frame) does not contain more groups than a specified parameter relative to the length of the data frame. If no checks were performed this would be highly disclosive, for example if the number of groups was the same as the number of rows, this would return the entire column of participant data.

5. We integrate this package with DataSHIELD disclosure control options that can be set by data owners. This enables data owners to permit or block certain collections of functions depending on the level of privacy security required. For example, dsFilter could be vulnerable to subsetting attacks, so it is blocked in the 'avocado' mode (designed to prevent such attacks), but permitted in other privacy modes.

3. Examples

To illustrate the improvements brought about by dsTidyverse, we provide three examples using the well-known 'mtcars' dataset. Each example contrasts the approach using dsBaseClient with the streamlined alternative using dsTidyverse-Client.

Example 1: Recoding a continuous variable as categorical

We return to the example provided in the introduction, of recoding the continuous variable mpg (miles per gallon) into four fuel efficiency categories:

- 0: <15 (very low)
- 1: 15–20 (low)
- 2: 20–25 (moderate)
- 3: >25 (high)

We previously saw how performing this operation with dsBaseClient was quite verbose. Using dsTidyverseClient, this is achieved in a single call:

```
ds.case_when(tidy_expr = list (mtcars$mpg < 15 ~ 0, mtcars$mpg >= 15 & mtcars$mpg <
20 ~ 1, mtcars$mpg >= 20 & mtcars$mpg < 25 ~ 2, mtcars$mpg >= 25 ~ 3), newobj =
"mpg_category")
```

Example 2: Creating a subset of columns

We want to retain only the columns 'mpg', 'cyl', 'hp', 'wt', and 'gear'. Using dsBaseClient requires identifying column indices and creating a subset:

```
ds.colnames("mtcars") ("mpg"  "cyl"  "disp" "hp"   "drat" "wt"   "qsec" "vs"   "am"
"gear" "carb")
ds.dataFrameSubset (df.name = "mtcars", V1 = "id_var", V2 = "id_var", Boolean.
operator = "==", keep.cols = c("1", "2", "4", "6", "10"), newobj =
"subset_mtcars")
```

Using dsTidyverseClient, this is greatly simplified:

```
ds.select (df.name = "mtcars", tidy_expr = list (mpg, cyl, hp, wt, gear), newobj =
"subset_mtcars")
```

Example 3: Filtering on multiple conditions

We create a subset where cars have:

- More than 6 cylinders
- Horsepower greater than 150
- Weight (wt) less than 3.5

Using dsBaseClient, this requires chaining three calls:

```
ds.dataFrameSubset (df.name = "mtcars", V1 = "cyl", V2 = "6", Boolean.operator =
">", newobj = "step1")
ds.dataFrameSubset (df.name = "step1", V1 = "hp", V2 = "150", Boolean.operator =
">", newobj = "step2")
ds.dataFrameSubset (df.name = "step2", V1 = "wt", V2 = "3.5", Boolean.operator =
"<", newobj = "filtered_mtcars")
```

Using dsTidyverseClient, the same logic is done in one line:

```
ds.filter (df.name = "mtcars", tidy_expr = list (cyl > 6 & hp > 150 & wt < 3.5),
newobj = "filtered_mtcars")
```

These three examples highlight how dsTidyverseClient reduces both code complexity and time investment for common data manipulation tasks. Further use cases and advanced patterns are provided in the package vignette.

4. Summary

In this paper we have illustrated the development of dsTidyverseClient, a DataSHIELD implementation of selected tidyverse functions. We hope that this package will provide researchers with more flexible and powerful tools for data manipulation and greatly improve the user experience of DataSHIELD.

5. Operation

To use dsTidyverse, the analyst must have:

- R version ≥4.4.0 installed locally
- The dsTidyverseClient package installed from CRAN
- An active DataSHIELD client-server infrastructure with the dsTidyverse package installed on the server side
- An active internet connection and authentication credentials for the federated environment

Full details of setting up DataSHIELD are provided in the DataSHIELD wiki (https://wiki.datashield.org/en/home).

Data availability

No data associated with this article. All vignettes in this paper use the 'mtcars' dataset, which is freely available with RStudio.

Software availablility

dsTidyverse is maintained as part of the long-running MOLGENIS open-source project for scientific software (https://molgenis.org/). Requests for the implementation of new functions are welcome, as are contributions from developers.

The packages are available to install from https://cran.r-project.org/web/packages/dsTidyverse/index.html and https://cran.r-project.org/web/packages/dsTidyverseClient/index.html

Source code is available from: https://github.com/molgenis/dsTidyverse and https://github.com/molgenis/dsTidyverse-Client

Archived source coded is available at https://doi.org/10.5281/zenodo.15462381.

The packages are licensed under LGPLv3.

References

Cadman T, Elhakeem A, Vinther JL, *et al.*: **Associations of Maternal Educational Level, Proximity to Green Space During Pregnancy, and Gestational Diabetes with Body Mass Index from Infancy to Early Adulthood: A Proof-of-Concept Federated Analysis in 18 Birth Cohorts.** *Am. J. Epidemiol.* 2024; **193**(5): 753–763.
PubMed Abstract | Publisher Full Text | Free Full Text

Doiron D, Burton P, Marcon Y, *et al.*: **Data Harmonization and Federated Analysis of Population-Based Studies: The BioSHaRE Project.** *Emerg. Themes Epidemiol.* 2013; **10**: 1–8.
Publisher Full Text

Escriba-Montagut X, Marcon Y, Anguita-Ruiz A, *et al.*: **Federated PrivacyProtected Meta-and Mega-Omics Data Analysis in Multi-Center Studies with a Fully Open-Source Analytic Platform.** *PLoS Comput. Biol.* 2024; **20**(12): e1012626.
PubMed Abstract | Publisher Full Text | Free Full Text

Gaye A, Marcon Y, Isaeva J, *et al.*: **DataSHIELD: Taking the Analysis to the Data, Not the Data to the Analysis.** *Int. J. Epidemiol.* 2014; **43**(6): 1929–1944.
PubMed Abstract | Publisher Full Text | Free Full Text

Jaddoe VWV, Felix JF, Andersen A-MN, *et al.*: **The LifeCycle Project-EU Child Cohort Network: A Federated Analysis Infrastructure and Harmonized Data of More Than 250,000 Children and Parents.** *Eur. J. Epidemiol.* 2020; **35**: 709–724.
PubMed Abstract | Publisher Full Text | Free Full Text

Knoppers BM, Harris JR, Tassé AM, *et al.*: **Towards a Data Sharing Code of Conduct for International Genomic Research.** *Genome Med.* 2011; **3**: 44–46.
Publisher Full Text

Mailund T, Mailund T: **"Tidy Evaluation." Domain-Specific Languages in R: Advanced Statistical Programming.** 2018; 135–157.

de Moira P, Angela SH, Strandberg-Larsen K, *et al.*: **The EU Child Cohort Network's Core Data: Establishing a Set of Findable, Accessible, Interoperable and Re-Usable (FAIR) Variables.** *Eur. J. Epidemiol.* 2021; **36**: 565–580.
PubMed Abstract | Publisher Full Text | Free Full Text

Vrijheid M, Basagaña X, Gonzalez JR, *et al.*: **Advancing Tools for Human Early Lifecourse Exposome Research and Translation (ATHLETE): Project Overview.** *Environ. Epidemiol.* 2021; **5**(5): e166.
PubMed Abstract | Publisher Full Text | Free Full Text

Wickham H, Averick M, Bryan J, *et al.*: **Welcome to the Tidyverse.** *J. Open Source Softw.* 2019; **4**(43): 1686.
Publisher Full Text

Open Peer Review

Current Peer Review Status: ✔ ✔

Version 1

Reviewer Report 06 November 2025

https://doi.org/10.5256/f1000research.180842.r419324

© **2025 Awe O.** This is an open access peer review report distributed under the terms of the Creative Commons Attribution License, which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited.

✔ **Olaitan I Awe**
African Society for Bioinformatics and Computational Biology, Cape Town, South Africa

The authors of the manuscript entitled, 'dstidyverse': An Implementation of TidyverseWithin the DataSHIELD Ecosystem, described a library or package named dstidyverse which is based on the R programming language. This would make data manipulation easier for users in the DataSHIELD ecosystem. Tidyverse is a popular data wragling library within the R ecosystem and now dsTidyverse implements some of those Tidyverse functions for data manipulation and integration into the DataSHIELD architecture. dstidyverse is open source and freely available and installable from CRAN. I was able to install dsTidyverse in my R version 4.5.0 environment. My minor comment is that the authors should provide examples in their R documentation and the methods should be described with a bit more detail. The package is also available on GitHub (https://github.com/molgenis/ds-tidyverse) thereby making the code findable and reproducible.

Is the rationale for developing the new software tool clearly explained?
Yes

Is the description of the software tool technically sound?
Yes

Are sufficient details of the code, methods and analysis (if applicable) provided to allow replication of the software development and its use by others?
Yes

Is sufficient information provided to allow interpretation of the expected output datasets and any results generated using the tool?
Yes

Are the conclusions about the tool and its performance adequately supported by the findings presented in the article?

Yes

Competing Interests: No competing interests were disclosed.

Reviewer Expertise: Bioinformatics

I confirm that I have read this submission and believe that I have an appropriate level of expertise to confirm that it is of an acceptable scientific standard.

Reviewer Report 13 August 2025

https://doi.org/10.5256/f1000research.180842.r395741

© 2025 Puskaric M. This is an open access peer review report distributed under the terms of the Creative Commons Attribution License, which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited.

Miroslav Puskaric
University of Stuttgart, Stuttgart, Germany

This article reports a data manipulation software package dstidyverse, which is already available as a part of the R Data SHIELD library. It addresses the issue of manipulating data such as renaming variables, defining subsets, or conditional formatting, which was prior possible through multiple steps, thus being an overhead for data scientists, even more if analyzing data from multiple sites.
Data SHIELD is popular in the health data analysis sector, where there are many use cases. The paper demonstrates the software functionalities on the dataset containing vehicle related information. As a further complement, it would be interested to explore any related work outside of health data.
A common use case is conducting pooled (federated) analyses across multiple sites, which can be particularly challenging when the data is not harmonized. Describing how this software package facilitates such scenarios would be valuable.
Many thanks to the authors for the great work, which will further improve workflows for the non-disclosive analysis of sensitive data.

Is the rationale for developing the new software tool clearly explained?
Yes

Is the description of the software tool technically sound?
Yes

Are sufficient details of the code, methods and analysis (if applicable) provided to allow replication of the software development and its use by others?
Yes

Is sufficient information provided to allow interpretation of the expected output datasets

and any results generated using the tool?
Yes

Are the conclusions about the tool and its performance adequately supported by the findings presented in the article?
Yes

Competing Interests: No competing interests were disclosed.

Reviewer Expertise: privacy enhancing technologies, management of sensitive data

I confirm that I have read this submission and believe that I have an appropriate level of expertise to confirm that it is of an acceptable scientific standard.

The benefits of publishing with F1000Research:

- Your article is published within days, with no editorial bias
- You can publish traditional articles, null/negative results, case reports, data notes and more
- The peer review process is transparent and collaborative
- Your article is indexed in PubMed after passing peer review
- Dedicated customer support at every stage

For pre-submission enquiries, contact research@f1000.com

F1000Research